

RUST · POSTGRESQL · PGRX

PgPulse

I moved my monitoring **inside** PostgreSQL.
It did not go well for a while.

Harshit Ruwali · github.com/HarshitRuwali/pgpulse

BEFORE WE START

I did not set out to write
a PostgreSQL extension.
I ended up writing one.

I am not an expert in PostgreSQL internals. Some choices here can be improved.

If you see a better way, I am open to suggestions and contributions.

The code is public: github.com/HarshitRuwali/pgpulse

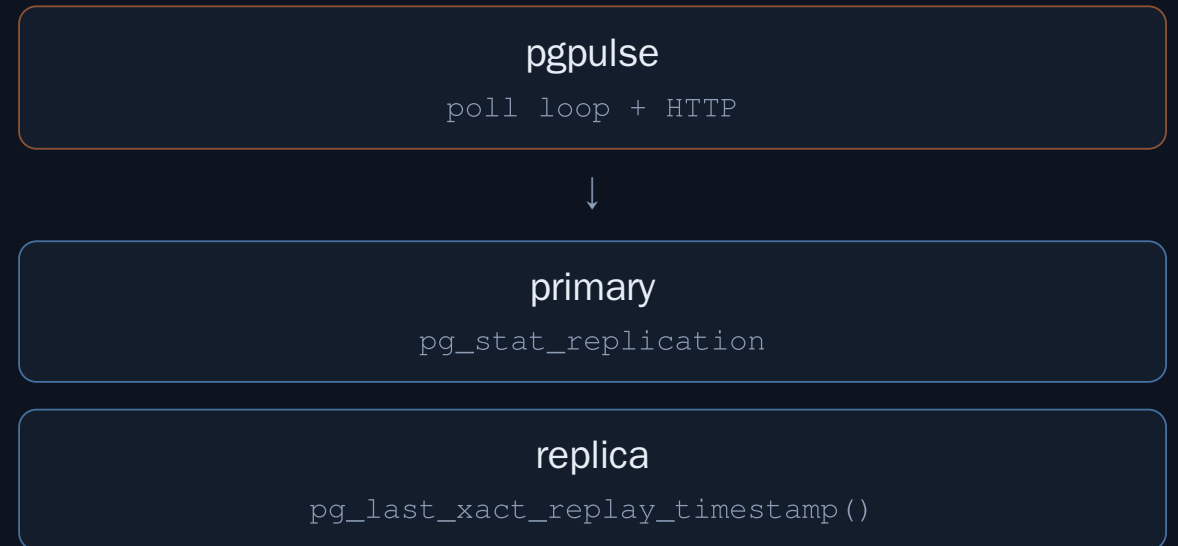
ONE

A monitor that needed monitoring

v1: a normal Rust service

- `tokio-postgres`
- `Arc<RwLock<Snapshot>>`
- `axum` + `prometheus`

Two weeks. Boring. Worked.



Now I had two things
to keep alive.

TWO

The database became the runtime

What an extension actually is

```
pgpulse.so      → $libdir  
pgpulse.control → extension/  
pgpulse--0.2.0.sql
```

postgresql.conf

```
shared_preload_libraries = 'pgpulse'
```

Three files, one config line, and **one symbol**.

pgrx does the C part

pgpulse/**Cargo.toml**

```
[lib]
crate-type = ["cdylib"]

[dependencies]
pgrx = "=0.18.0"
```

pgpulse/src/**lib.rs**

```
::pgrx::pg_module_magic!();

/// Return the health status
#[pg_extern]
fn pgpulse_health_status() -> String {
    let snapshot = shared_mem::read_snapshot();
    let health_status = format!("{:?}", snapshot.health_status);
    health_status
}
```

Write a Rust function, get a **real SQL function**. pgrx generates the C glue and the DDL.

How it actually boots

postmaster

reads `shared_preload_libraries`, `dlopen` `pgpulse.so`
checks the magic block first, or refuses: "incompatible library"

postmaster

`dlsym("_PG_init")` and calls it
by name. no registration, no manifest

`_PG_init`

`guc::init()` · `shared_mem::init()` · `bgw::init()`
define settings · install the shmem hooks · register the worker

postmaster

sizes shared memory, then allocates it
`RequestAddinShmemSpace(size_of::<Shared<T>>())` → `ShmemInitStruct`

fork

the worker is forked, and reloads the library
so `_PG_init` runs **again**, in the child

worker

`dlsym("pgpulse_worker_main")`, then the poll loop
attach signals · connect to SPI · every 10s forever

All of it before a single client has connected.

THREE

Four contracts, one loop

Four contracts to get right

GUC

Config

Background worker

Your loop

Shared memory

Your state

SPI

SQL from inside

Config → process → state → query

GUC

pgpulse/src/**guc.rs** · init()

```
pub static REPLICA_HOST: GucSetting<Option<CString>> = GucSetting::<Option<CString>>::new(None);

GucRegistry::define_string_guc(
    c"pgpulse.replica_host",
    c"host",
    c"Hostname or IP address of the replica PostgreSQL instance to monitor",
    &REPLICA_HOST,
    GucContext::Sighup, // takes effect on config reload
    GucFlags::default(),
);
```

postgresql.conf

```
pgpulse.replica_host = '192.168.1.20'
pgpulse.replica_port = 5432
```

My settings become **Postgres settings**. `SHOW` works, `pg_settings` works, reload works.

Background worker

pgpulse/src/bgw.rs · init(), called from `_PG_init`

```
BackgroundWorkerBuilder::new("postgres monitoring worker")
    .set_function("pgpulse_worker_main") // entry point, by name
    .set_library("pgpulse")
    .enable_spi_access()
    .set_restart_time(Some(Duration::from_secs(10))) // restart if it crashes
    .load();
```

pgpulse/src/bgw.rs · `pgpulse_worker_main()`, the entry point above

```
pub extern "C-unwind" fn pgpulse_worker_main(_arg: pg_sys::Datum) {
    BackgroundWorker::attach_signal_handlers(SignalWakeFlags::SIGHUP | SignalWakeFlags::SIGTERM);
    BackgroundWorker::connect_worker_to_spi(Some("postgres"), None);

    loop {
        let interval = Duration::from_secs(guc::POLL_INTERVAL_SECONDS.get() as u64);
        let alive = BackgroundWorker::wait_latch(Some(interval)); // sleep 10s
        if !alive {
            break; // SIGTERM, or postmaster died
        }
        ... // collect, then write
    }
}
```

Shared memory

pgpulse/src/shared_mem.rs

```
static PGPULSE_METRICS: PgLwLock<MetricSnapshot> = unsafe { PgLwLock::new(c"pgpulse_metrics") };

pub fn write_snapshot(snapshot: MetricSnapshot) {    // the worker, every 10s
    let mut guard = PGPULSE_METRICS.exclusive();
    *guard = snapshot;
}

pub fn read_snapshot() -> MetricSnapshot {          // any user session, via SQL
    let guard = PGPULSE_METRICS.share();
    guard.clone()
}
```

One writer, many readers, one lock. The only state the extension keeps.

SPI

pgpulse/src/collectors/**replication.rs** · collect_replication_clients()

```
SELECT
    application_name,
    state,
    EXTRACT(EPOCH FROM replay_lag)::float8 AS replay_lag_seconds,
    pg_wal_lsn_diff(
        pg_current_wal_lsn(),
        COALESCE(replay_lsn, flush_lsn, write_lsn)
    )::bigrint AS lsn_gap_bytes
FROM pg_stat_replication
```

```
Spi::connect(|client| {
    let rows = client.select(query, None, &[])?;
    ...
})
```

The primary's own view of every connected replica, read without a connection.

One poll. Two views. One verdict.

PGPULSE_WORKER_MAIN · EVERY 10 SECONDS BY DEFAULT



FOUR

Three normal Rust assumptions broke

ASSUMPTION 1

Shared memory vs the heap

WHAT I WROTE

pgpulse/src/models.rs

```
pub struct MetricSnapshot {
    pub replication_metrics: ReplicationMetrics,
    pub primary_metrics: PrimaryMetrics,           // holds a Vec<ReplicationClient>
    pub health_status: HealthStatus,
    pub collected_at: DateTime<Utc>,
    pub long_running_queries: Vec<LongRunningQueries>,
}

unsafe impl pgrx::PGRXSharedMemory for MetricSnapshot {}
```


ASSUMPTION 2

The background worker

WHAT I WROTE

pgpulse/src/bgw.rs · collect_and_store_metrics()

```
pub fn collect_and_store_metrics() -> anyhow::Result<MetricSnapshot> {  
    let (replication_clients, replication_client_count) =  
        replication::collect_replication_clients()  
            .map_err(|e| anyhow::anyhow!("SPI error collecting replication clients: {e:?}"))?;  
  
    let long_running_queries = queries::get_long_running_queries()  
        .map_err(|e| anyhow::anyhow!("SPI error collecting long-running queries: {e:?}"))?;  
    ...  
}
```

Two SPI calls. Looks like any other function.

WHAT HAPPENED

```
LOG:  background worker "postgres monitoring worker" (PID 62) was terminated by signal 11: Segmentation fault
LOG:  background worker "postgres monitoring worker" (PID 74) was terminated by signal 11: Segmentation fault
LOG:  background worker "postgres monitoring worker" (PID 91) was terminated by signal 11: Segmentation fault
```

THE FIX

```
pgpulse/src/bgw.rs · collect_and_store_metrics()
```

```
+ // SPI requires an active transaction in a BGW
+ BackgroundWorker::transaction(|| {
- let (replication_clients, replication_client_count) =
-     replication::collect_replication_clients()
+     let (replication_clients, _replication_client_count) =
+         replication::collect_replication_clients()
+
+     ...
- Ok(snapshot)
+     Ok(snapshot)
+ })
```

Backends get a transaction for free. Workers get nothing.

You are a C library, not a crate

pgpulse/src/bgw.rs

```
.set_function("pgpulse_worker_main")    // Postgres will dlsym this string

#[pg_guard]
#[no_mangle]                            // ...so don't let Rust rename the symbol
pub extern "C-unwind" fn pgpulse_worker_main(_arg: pg_sys::Datum) { ... }
```

pgpulse/src/lib.rs · _PG_init()

```
// `MyBgworkerEntry` is non-NULL inside every forked BGW process and NULL elsewhere.
extern "C" {
    static MyBgworkerEntry: *const pg_sys::BackgroundWorker;
}
let in_bgw = !unsafe { MyBgworkerEntry }.is_null();
if !in_bgw {
    bgw::init();
}
```

It calls you **by name**, and it runs `_PG_init()` **again** in the worker.

ASSUMPTION 3

I reached for a client library

WHAT I WROTE

pgpulse/**Cargo.toml** · and collectors/replication.rs

```
postgres = "0.19.13"
```

```
let mut client = Client::connect(&conn_str, NoTls).ok()?;  
let row = client.query_one(  
    "SELECT EXTRACT(EPOCH FROM (now() - pg_last_xact_replay_timestamp()))::float8", &[])  
    .ok()?;
```

The same client I used in v1. One query. What could go wrong.

WHAT HAPPENED

A backend is **single-threaded by contract**. `postgres` is a blocking wrapper over `tokio-postgres`: sync API, still a runtime, still threads.

THE FIX

pgpulse/src/collectors/**replication.rs** · `collect_replica_time_lag()`

```
- // let mut client = Client::connect(&conn_str, NoTls).ok()?;
- // let row = client.query_one("SELECT EXTRACT(EPOCH FROM ...", &[]).ok()?;
- None
+ let conn: *mut PGconn = PQconnectdb(conn_str.as_ptr());
+ if PQstatus(conn) != ConnStatusType::CONNECTION_OK {
+     PQfinish(conn);
+     return None;
+ }
+ let result = PQexec(conn, query.as_ptr());
+ ...
+ PQclear(result);
+ PQfinish(conn);
```

That `None` shipped. For two weeks.

AFTER THE BIG THREE

The smaller cuts

interval out of range

```
- AND now() - query_start > make_interval(
+ AND EXTRACT(EPOCH FROM (now() - query_start)) > 10
```

Capacity is not length

```
- clients[count] = ReplicationClient {
+ clients.push(replica_client).ok();
```

GUCs on SIGHUP

```
if BackgroundWorker::sighup_received() {
-   guc::init();
+   // No need to re-init the GUCs again
}
```

Also `connect_timeout=5`, because libpq's default is **minutes**.

THE PATTERN UNDERNEATH THEM

The manual was one layer down

Where I actually found answers:

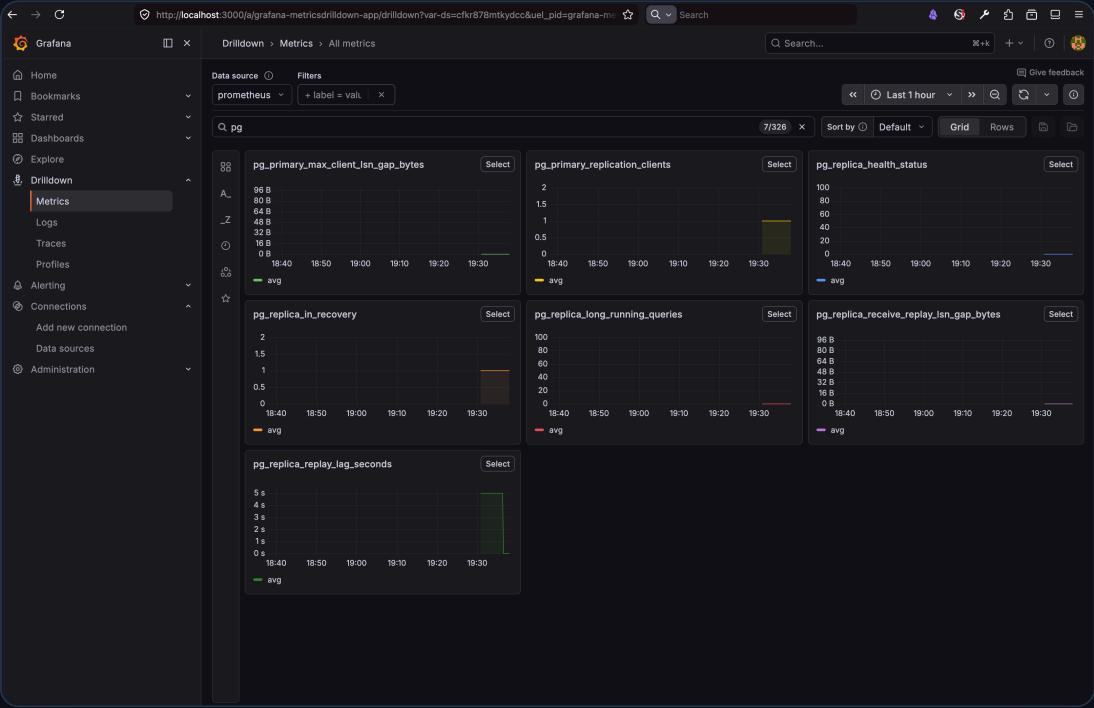
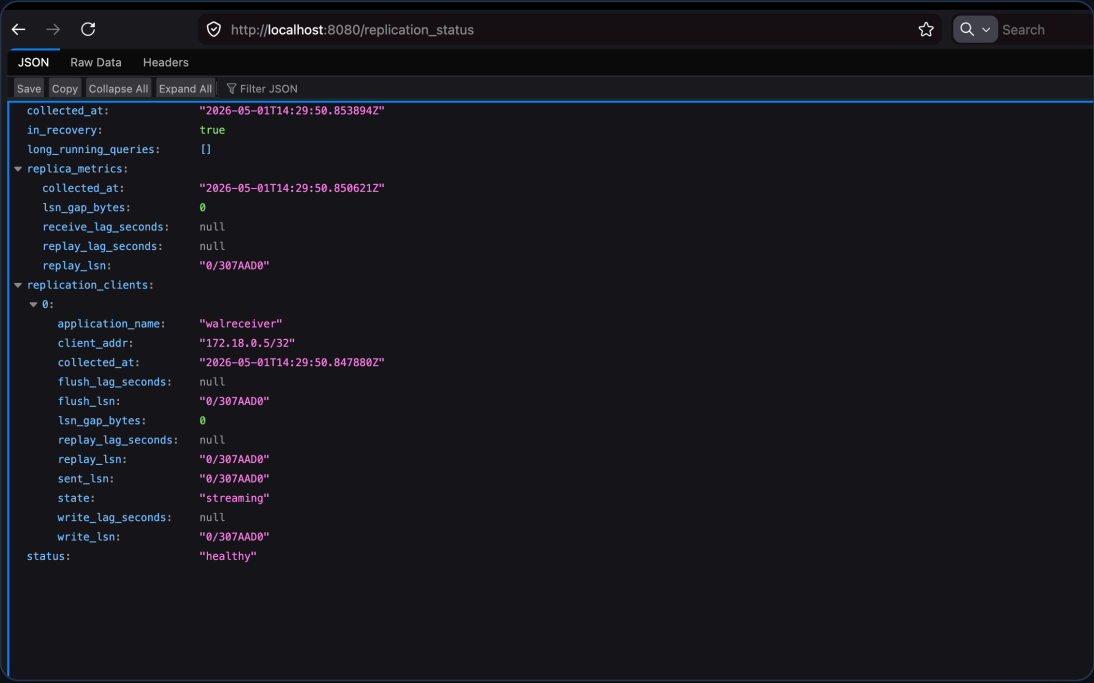
- the pgrx source code
- `examples/`, read as documentation
- Twitch videos from six years ago
- slides from a FOSDEM 2025 talk

There is an API reference.

There is no guide.

DEMO

Pause the replica. Watch lag climb.



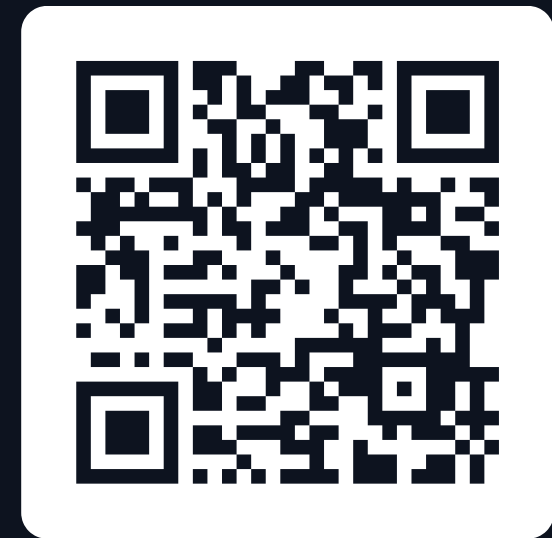
Thanks

github.com/HarshitRuwali/pgpulse

blog.sp00fexpl01t.rocks

x.com/harshitruwali

Questions



REACH OUT